

Salary Queries

A company has n employees with certain salaries.
Your task is to keep track of the salaries and process queries.

Input

The first input line contains two integers n and q: the number of employees and queries.
The employees are numbered 1,2,...,n.

The next line has n integers p_1,p_2,...,p_n: each employee's salary.
After this, there are q lines describing the queries.

Each line has one of the following forms:

- ! k x: change the salary of employee k to x
- ? a b: count the number of employees whose salary is between a ... b

Output

Print the answer to each ? query.

Constraints

- 1 ≤ n, q ≤ 2 · 10^5
- 1 ≤ p_i ≤ 10^9
- 1 ≤ k ≤ n
- 1 ≤ x ≤ 10^9
- 1 ≤ a ≤ b ≤ 10^9

Example

Input:

5 3
3 7 2 2 5
? 2 3
! 3 6
? 2 3

Output:

3
2

We have:

- n employees
- Each employee has one salary
- Salaries change over time

Queries:

- ! k x → update salary of employee k
- ? a b → count employees with salary in [a, b]

Key observation:

We are not asked for sums, only counts.

So this is a frequency problem, not a value problem.

Before thinking about Fenwick Tree, ask:

"What exactly am I querying?"

Answer:

How many salaries lie in a value range [a, b]

This is not index-based, it is value-based.

Why a Normal Array / Fenwick Tree Doesn't Work Directly

Salaries:

1 ≤ salary ≤ 1e9

You cannot do:

freq[salary]++

Because:

- Memory would explode
- Fenwick Tree needs indices in a dense range

So you think:

"I don't care about actual salary values,
I only care about their relative order."

This thought leads directly to...

Coordinate Compression

What compression really means

You replace large values with small ranks, while preserving order.

Example salaries:

[3, 7, 2, 2, 5]

Sorted unique:

[2, 3, 5, 7]

Mapping:

2 → 1
3 → 2
5 → 3
7 → 4

Now everything fits in [1 ... n].

What Will Fenwick Tree Store?

After compression, think in frequencies.

Fenwick Tree will store:

bit[i] = how many employees currently have salary = compressed_value_i

So:

- Update = remove old salary, add new salary
- Query = count frequencies in a compressed range

Code

```
#include <bits/stdc++.h>
using namespace std;

struct Fenwick {
    int n;
    vector<int> bit;

    Fenwick(int n) : n(n), bit(n + 1, 0) {}

    void update(int idx, int delta) {
        while (idx <= n) {
            bit[idx] += delta;
            idx += idx & -idx;
        }
    }

    int query(int idx) {
        int sum = 0;
        while (idx > 0) {
            sum += bit[idx];
            idx -= idx & -idx;
        }
        return sum;
    }

    int rangeQuery(int l, int r) {
        return query(r) - query(l - 1);
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    cin >> n >> q;

    vector<int> p(n);
    for (int i = 0; i < n; i++) {
        cin >> p[i];
    }

    vector<tuple<char, int, int>> queries;
    vector<int> values;

    // Initial salaries
    for (int x : p) values.push_back(x);

    for (int i = 0; i < q; i++) {
        char type;
        cin >> type;
        if (type == '!') {
            int k, x;
            cin >> k >> x;
            queries.emplace_back(type, k - 1, x);
            values.push_back(x);
        } else {
            int a, b;
            cin >> a >> b;
            queries.emplace_back(type, a, b);
            values.push_back(a);
            values.push_back(b);
        }
    }

    // Coordinate compression
    sort(values.begin(), values.end());
    values.erase(unique(values.begin(), values.end()), values.end());

    auto getId = (int x) {
        return lower_bound(values.begin(), values.end(), x) - values.begin() + 1;
    };

    Fenwick fw(values.size());

    // Initialize frequencies
    for (int i = 0; i < n; i++) {
        fw.update(getId(p[i]), 1);
    }

    // Process queries
    for (auto &[type, x, y] : queries) {
        if (type == '!') {
            // remove old salary
            fw.update(getId(p[x]), -1);
            // add new salary
            p[x] = y;
            fw.update(getId(p[x]), +1);
        } else {
            int left = getId(x);
            int right = getId(y);
            cout << fw.rangeQuery(left, right) << "\n";
        }
    }

    return 0;
}
```